

PostgreSQL Scaling Lab

Deep Dive

How far a single relational database goes
before you replace it

What you are holding

An exhaustive, beginner-safe walkthrough of a three-part study: find slow queries and index them, partition a time-series table, then add streaming read replicas with read/write routing in the application.

Assumed knowledge: none. Every technical term is defined the first time it appears. Every performance number is labelled with where it came from and whether it was re-measured while writing this document.

Contents

1	What this project is	2
1.1	What is the author's work, and what was handed to him	2
2	Ground floor: what a database is actually doing	3
2.1	Rows live in blocks	3
2.2	Without an index, the only option is to read everything	3
2.3	An index is a sorted map to the rows	4
2.4	The planner decides, and it can decide not to use your index	5
2.5	EXPLAIN is how you see any of this	5
2.6	Shared buffers, and why the first read is a lie	6
3	The shape of the repository	7
4	Part 1: finding slow queries, and indexing them	8
4.1	First, make the database tell on itself	8
4.2	The data	9
4.3	The load generator, and the "black box" claim	10
4.4	The solution: seven indexes	11
4.5	Which index actually serves which real query	13
4.6	When GIN loses	15
4.7	What part 1 costs	16
5	Part 2: partitioning	16
5.1	The idea	16
5.2	The data and the strategy	17
5.3	The migration has three real defects	18
5.4	What the benchmarks actually show	19
5.5	What part 2 genuinely establishes	21
6	Part 3: read replicas	22
6.1	The idea	22
6.2	How PostgreSQL copies a server	22
6.3	Bootstrapping a replica inside Compose	23
6.4	The credential story, and where it does not hold	24
6.5	The routing application	25
6.6	Replication lag, the bug you cannot fix by trying harder	26
6.7	The tests, and why they cannot fail	26
7	What was actually verified, and what was not	27
8	Consolidated honest findings	28
9	If you are picking this up again	29

1 What this project is

A course assignment sets up a story. A company runs everything on PostgreSQL, a relational database. Queries are getting slow. The CTO has heard that NoSQL is “infinitely scalable” and wants to migrate. The engineer’s job is to show how much performance is still available inside the database they already have, using the standard toolbox, before throwing it away.

The project answers that in three parts, in escalating order of cost and disruption:

1. **Part 1, indexing.** Turn on slow-query logging, find out which queries are slow and why, and build the right index for each. Nothing about the architecture changes.
2. **Part 2, partitioning.** Split one large time-series table into many physical pieces so that a query about March only touches March. The table’s shape changes; the server does not.
3. **Part 3, read replicas.** Add two more servers that continuously copy the first one, and teach the application to send writes to the original and reads to the copies. Now there are three machines and a new class of bug.

Jargon: Relational database, RDBMS

Software that stores data in tables (rows and columns), enforces structure, and answers questions written in SQL. PostgreSQL is one. “RDBMS” stands for Relational Database Management System. The alternative the CTO wanted, “NoSQL”, is a loose family of databases that give up some of that structure and some guarantees in exchange for easier spreading of data across many machines.

Jargon: SQL

Structured Query Language. The language you use to ask a relational database for data. `SELECT * FROM users WHERE id = 5` means “give me every column of the row in the users table whose id is 5”.

1.1 What is the author’s work, and what was handed to him

This matters for reading the rest of the document honestly. The repository is a mix of course-provided scaffolding and the author’s own work.

Provided by the course	Written by the author
The assignment briefs (<code>docs/ASSIGNMENT.md</code> , <code>part_1/README.md</code> , <code>part_2/README.md</code> , <code>part_3/README.md</code>)	The three <code>solution.sql</code> / solution files
The table definitions (<code>init.sql</code> in each part)	The whole replication setup in part 3: <code>compose changes</code> , <code>primary.conf</code> , <code>replica1.conf</code> , <code>replica2.conf</code> , <code>pg_hba.conf</code>
The baseline single-node application <code>part_3/app/app.py</code>	The routing application <code>part_3/app/solution.py</code>
The test suite <code>part_3/app/tests/test_events_db.py</code>	All analysis documents
The load generator binaries in <code>part_1/load_tester/</code>	

Table 1: Authorship split, as stated in the project README and consistent with the `part_3/diff.patch` file, which records the author’s changes against the original scaffolding.

The interesting engineering is therefore concentrated in three small files (`part_1/solution.sql`, `part_2/solution.sql`, `part_3/app/solution.py`), a handful of PostgreSQL configuration files, and

the reasoning about what to measure. That is a good thing to know before an interview: the volume of code is small, so the defense has to be the reasoning.

2 Ground floor: what a database is actually doing

Everything in this project is a consequence of a few mechanical facts about how PostgreSQL stores and finds rows. If these are clear, all three parts are obvious. If they are not, no amount of tuning advice will make sense.

2.1 Rows live in blocks

PostgreSQL does not store a table as a list of rows floating in memory. It stores it as a file on disk, cut into fixed-size chunks called **blocks** (also called pages), each 8 kilobytes by default. Rows are packed into blocks. The whole file is called the **heap**.

Jargon: Block (page)

The unit of I/O. PostgreSQL never reads half a block. If you need one row, the server reads the entire 8 KB block that contains it. This is why the number of blocks a query touches is the most honest measure of how much work it did.

Jargon: Heap

The main file holding a table's rows, in no particular order. New rows go wherever there is space. "Heap" here means "unordered pile", not the heap data structure from a programming course.

The `users` heap: 100,000 rows packed into 2,494 blocks of 8 KB

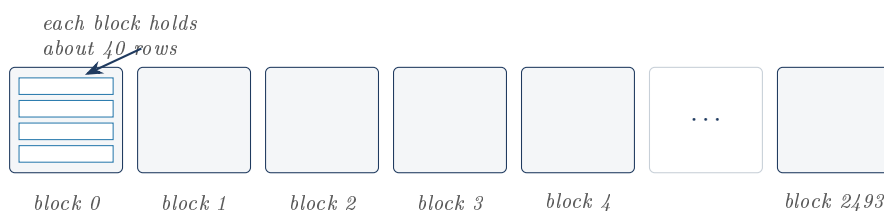


Figure 1: A table is a pile of fixed-size blocks. The block counts here are the real, measured ones for this project's `users` table.

2.2 Without an index, the only option is to read everything

Suppose you ask for the user whose email is `user50000@example.com`, and the database has no help. It has no idea where that row is. The rows are in no order. So it does the only thing it can: read block 0, check every row in it, read block 1, check every row, and so on to the end. This is a **sequential scan**.

Jargon: Sequential scan (seq scan)

Read every block of the table and test every row against the condition. The cost is proportional to the size of the whole table, no matter how few rows match. Finding one row out of 100,000 costs the same as finding all of them.

That is not a hypothetical. Measured on this project’s own data, that exact query reads all 2,494 blocks and throws away 99,999 of the 100,000 rows it looked at. The work is almost entirely waste, and the waste grows with the table.

2.3 An index is a sorted map to the rows

An **index** is a second, separate structure that stores the values of one column (or an expression) in sorted order, each paired with a pointer to where that row lives in the heap. Because it is sorted, the database can find a value in it by repeated halving instead of by reading it all.

Jargon: Index

A separate on-disk structure that maps column values to row locations, kept in an order that makes lookups cheap. It is not a copy of the table and it is not free: every **INSERT**, **UPDATE** and **DELETE** must also update every index on the table, and each index consumes disk.

Jargon: B-tree

The default index type, and the one most people mean when they say “index”. A balanced tree of blocks. The top block splits the whole range of values into a few bands and says which block to visit for each band; those blocks split their band further; the bottom level holds the actual values and the pointers to the rows. Each level you descend is one block read, and the tree is very shallow (three or four levels covers millions of rows), so a lookup costs a handful of block reads instead of thousands. “B” is for balanced: every leaf sits at the same depth, so no value is slower to find than any other.

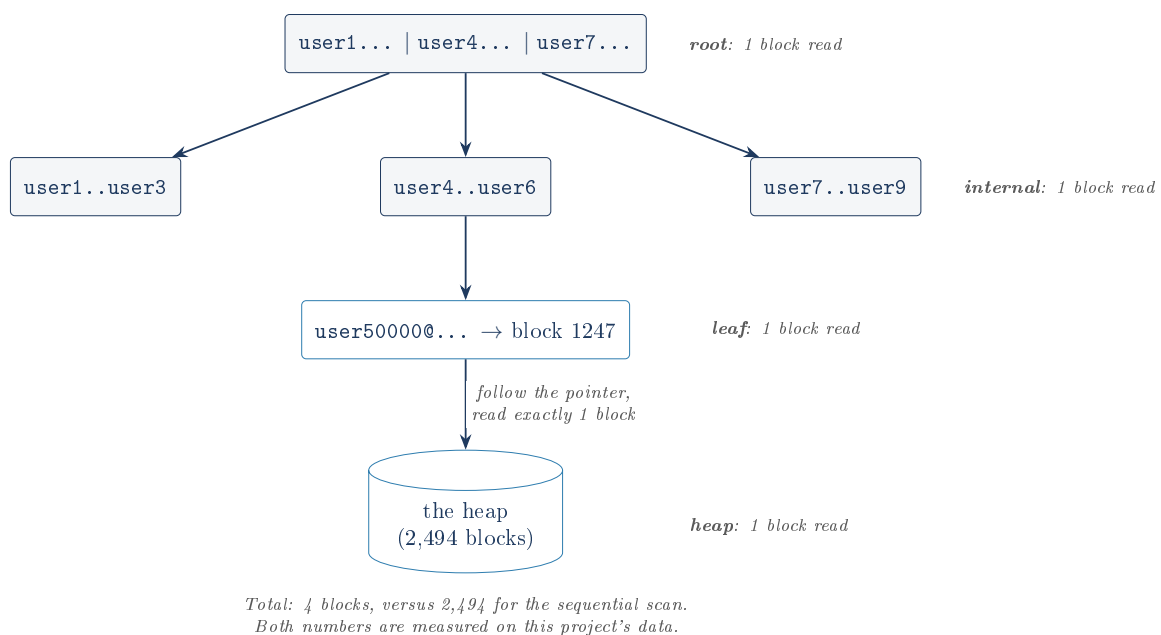


Figure 2: A B-tree turns “search everywhere” into “descend a few levels”. The 4-block figure is the real measurement recorded in `docs/screenshots/run-output.txt` and independently reproduced while writing this document.

The whole of Part 1 in one sentence

2,494 blocks becomes 4 blocks. That is the entire value proposition of indexing, and every remaining subtlety is about *which* index structure can answer *which* question.

2.4 The planner decides, and it can decide not to use your index

You do not tell PostgreSQL how to run a query. You tell it what you want, and the **query planner** picks a strategy. It estimates the cost of each option using statistics about the data and chooses the cheapest estimate.

Jargon: Query planner (optimizer)

The component that turns “what I want” into “how to get it”. It considers plans (sequential scan, index scan, various join strategies), estimates a cost for each using collected statistics, and runs the cheapest. It can be wrong, and it can rationally refuse to use an index.

This last point is the one beginners find surprising and it recurs throughout this project. If a query matches most of the table, using the index is *slower* than just reading the table: you would pay for the index descent *and* then jump around the heap in random order, when reading the blocks in order would have been cheaper. The planner knows this. So an index you created can sit unused, and that is often the planner being right rather than the planner being broken.

Jargon: ANALYZE, and statistics

PostgreSQL keeps a sample-based summary of each column: roughly how many distinct values, which values are most common, how they spread. The **ANALYZE** command refreshes it. Without it the planner is guessing, and it will make bad choices. This turns out to matter for one of this project’s headline numbers, discussed in Section 5.4.

2.5 EXPLAIN is how you see any of this

EXPLAIN shows the plan the planner chose. **EXPLAIN ANALYZE** actually runs the query and shows what really happened. Adding **BUFFERS** reports how many blocks were touched.

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT * FROM users WHERE email = 'user50000@example.com';
3
4 Seq Scan on users  (cost=0.00..2712.22 rows=87 width=1134)
5                   (actual time=23.950..62.589 rows=1.00 loops=1)
6   Filter: ((email)::text = 'user50000@example.com'::text)
7   Rows Removed by Filter: 99999
8   Buffers: shared read=2494 dirtied=2494
9   Execution Time: 62.647 ms

```

Listing 1: Reading an EXPLAIN. This is real output from this project’s data, before any index existed.

Line by line, because this is the single most important skill the project teaches:

- **Seq Scan on users**: the strategy chosen. It is reading the whole table.
- **cost=0.00..2712.22**: the planner’s *estimate*, in arbitrary units. The first number is the cost to return the first row, the second to return them all. These are not milliseconds and mean nothing in absolute terms; they only exist to be compared against other plans.
- **rows=87**: the planner *guessed* 87 rows would match.
- **actual ... rows=1.00**: one row actually matched. The guess was 87x off, because without an index on the column PostgreSQL had poor information about it.
- **Rows Removed by Filter: 99999**: the waste, stated outright.
- **Buffers: shared read=2494**: 2,494 blocks were read from disk.
- **dirtied=2494**: it also *modified* all 2,494 blocks. This is a real subtlety covered in Section 2.6.

Blocks beat milliseconds

Milliseconds depend on cache warmth, on what else the machine is doing, and on the hardware. Block counts do not: they are a deterministic property of the plan and the data. Whenever this project has both numbers, the block count is the one to quote. The author reached this conclusion independently and recorded it in the repository; it is correct and it is the mark of someone who has actually benchmarked something.

2.6 Shared buffers, and why the first read is a lie

PostgreSQL caches blocks in memory in an area called **shared buffers**. An `EXPLAIN (BUFFERS)` line distinguishes `hit` (found in cache) from `read` (fetched from the operating system, possibly from disk).

The `dirty=2494` above deserves its own explanation, because it is the reason a naive first benchmark is untrustworthy. When a row is first written, PostgreSQL does not yet know whether the transaction that created it committed. The first reader to come along works it out and records the answer as a **hint bit** on the row, so nobody has to work it out again. Setting those bits modifies the block, which means the first scan of a freshly loaded table is also a large write.

The repository documents this honestly: the first-ever scan measured anywhere from 19 ms to 140 ms across trials, while the steady-state scan is a stable 15 to 21 ms. The author's conclusion, recorded in `docs/screenshots/run-output.txt`, is that a single sample of a first touch is not a benchmark. That is right, and it is a more sophisticated point than the assignment asked for.

3 The shape of the repository

```

postgres-scaling-lab
├── README.md project overview, results, honest limits
├── PROJECT_WALKTHROUGH.md older narrative walkthrough
├── run.sh the walkthrough's commands, as a script
├── .env.example
├── docs
│   ├── ASSIGNMENT.md the original brief
│   ├── EXPLANATION.md 1,494-line full write-up
│   ├── screenshots
│   │   ├── run-output.txt captured real benchmark stdout
│   │   └── app.png
│   └── explainers this document
├── part_1 indexing
│   ├── docker-compose.yml DB on :5433 + load tester
│   ├── init.sql 100k users, 500k orders, no indexes
│   ├── solution.sql the 7 indexes (author's work)
│   ├── INDEXING_ANSWERS.md
│   └── load_tester
│       ├── Dockerfile
│       ├── Makefile
│       ├── load_tester_amd64 prebuilt Go binary, no source
│       └── load_tester_arm64
├── part_2 partitioning
│   ├── docker-compose.yml DB on :5435
│   ├── init.sql 600k events + 3 indexes
│   ├── solution.sql parent table, 16 partitions, migration
│   └── PARTITIONING_ANSWERS.md
├── part_3 read replicas
│   ├── docker-compose.yml primary :5436, replicas :5437/:5438
│   ├── init.sql slots + events table + 1k rows
│   ├── primary.conf
│   ├── replica1.conf
│   ├── replica2.conf
│   ├── pg_hba.conf
│   ├── diff.patch record of changes vs scaffolding
│   ├── REPLICA_SETUP.md
│   └── app
│       ├── app.py course baseline, single node
│       ├── solution.py routing version (author's work)
│       ├── requirements.txt
│       └── tests/test_events_db.py course-provided, 5 tests

```

Figure 3: The repository. Three self-contained stacks plus documentation.

Each part is an independent Docker Compose stack on its own port, deliberately: they are three separate experiments, not one system. The README instructs you to run `docker compose down` between parts.

Jargon: Docker, container, Docker Compose

A **container** is an isolated packaged process with its own filesystem, so “PostgreSQL 17” runs identically on any machine without being installed on it. **Docker** runs containers. **Docker Compose** reads a `docker-compose.yml` file describing several containers and their relationships, and starts them together. Here it is what makes a three-node database cluster something you can create and destroy with one command.

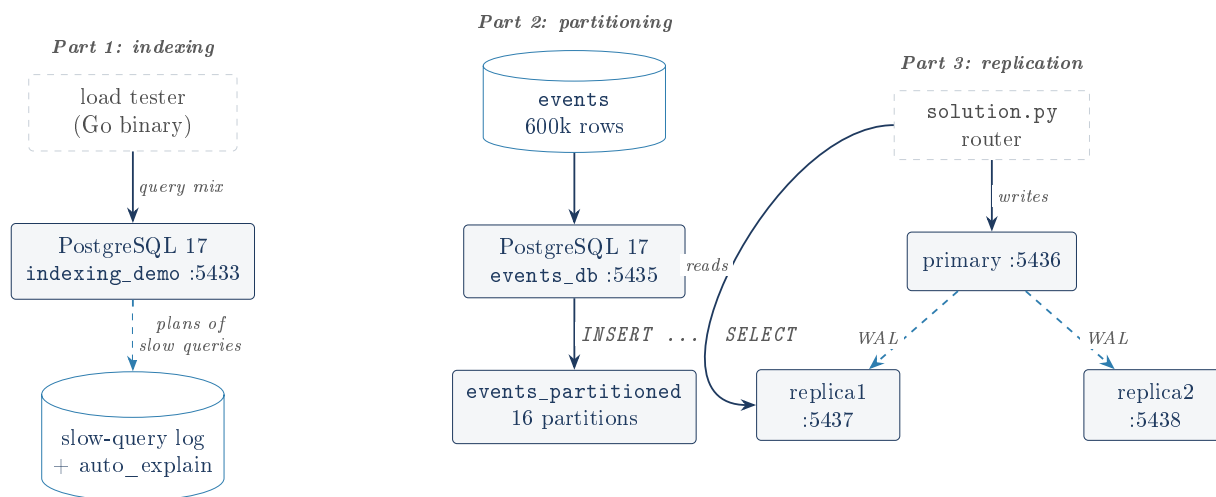


Figure 4: The three stacks. They never run at the same time and share no data. Part 2’s `events` table and part 3’s `events` table are different schemas that happen to share a name.

4 Part 1: finding slow queries, and indexing them

4.1 First, make the database tell on itself

You cannot fix what you cannot see. Before touching a single index, part 1 configures PostgreSQL to report its own slow queries. This happens in `part_1/docker-compose.yml`, in the command passed to the server.

```

1  command: [
2    "postgres",
3    "-c", "log_min_duration_statement=5",
4    "-c", "log_statement=none",
5    "-c", "shared_preload_libraries=auto_explain",
6    "-c", "auto_explain.log_min_duration=5",
7    "-c", "auto_explain.log_analyze=on",
8    "-c", "auto_explain.log_buffers=on",
9    "-c", "auto_explain.log_timing=on"
10 ]

```

Listing 2: `part_1/docker-compose.yml`, the observability configuration

Jargon: log_min_duration_statement

Log the text of any statement that took longer than this many milliseconds. Set to 5 here. In production you would set it far higher (a second or more); 5 ms is a teaching threshold chosen to make the interesting queries surface quickly.

Jargon: auto_explain

A bundled extension that logs not just the slow statement but its full execution plan, automatically, as it happens. Without it you would see “this query took 300 ms” and still have to reproduce it by hand to learn why. With `auto_explain.log_analyze=on` the logged plan includes real row counts and real timings; with `log_buffers=on` it includes block counts.

Jargon: shared_preload_libraries

Some extensions must be loaded into the server process at startup, before any client connects, because they hook into query execution itself. `auto_explain` is one. This is why it appears in the startup command rather than being enabled with a `CREATE EXTENSION` statement later.

The real lesson of part 1 is not “add indexes”

It is: make the database produce evidence, read the evidence, and let it tell you which index to build. The indexes are the easy part. The observability setup is the part that generalizes to any real system.

4.2 The data

`part_1/init.sql` builds two tables with no indexes at all (beyond the primary keys that come free) and fills them.

```

1 CREATE EXTENSION IF NOT EXISTS pg_trgm;
2
3 CREATE TABLE users (
4     id SERIAL PRIMARY KEY,
5     email VARCHAR(255),
6     full_name VARCHAR(255),
7     status VARCHAR(20),
8     metadata JSONB,
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );
11
12 CREATE TABLE orders (
13     id SERIAL PRIMARY KEY,
14     user_id INTEGER,          -- note: no FOREIGN KEY constraint
15     amount DECIMAL(10,2),
16     status VARCHAR(20),
17     items JSONB,
18     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
19 );
20
21 INSERT INTO users (email, full_name, status, metadata)
22 SELECT 'user' || n || '@example.com',
23        'User ' || n,
24        CASE WHEN n % 100 = 0 THEN 'inactive' ELSE 'active' END,
25        jsonb_build_object(
26            'last_login', NOW() - (n || ' days')::INTERVAL,
27            'preferences', jsonb_build_object(
28                'theme', (ARRAY['light','dark'])[1 + (n % 2)],
29                'language', (ARRAY['en','es','fr'])[1 + (n % 3)])
30 FROM generate_series(1, 100000) n;
```

Listing 3: `part_1/init.sql`, abridged

Several details here are load-bearing later:

- `generate_series(1, 100000)` is PostgreSQL’s built-in row generator. It is how the whole dataset is synthesized with no external tooling.

- `status` is `'inactive'` for exactly every hundredth user. So the split is 99,000 active to 1,000 inactive. **Verified by running it.** This ratio matters enormously and is misdescribed in the project's own notes; see Section 4.5.1.
- `theme` alternates `light/dark`, so a search for `theme = 'dark'` matches exactly half the table. This also matters; see Section 4.6.
- `orders.user_id` has no `FOREIGN KEY` declaration. It is a foreign key in intent only.
- `JSONB` is PostgreSQL's binary JSON type: a whole document stored in one column, queryable.

Jargon: JSONB and containment (@>)

`JSONB` stores a JSON document in a parsed binary form. The `@>` operator means “contains”: `metadata @> '{"theme":"dark"}'` is true for any document that includes that key and value, at any depth from the top. It is the operator a GIN index can accelerate.

4.3 The load generator, and the “black box” claim

`part_1/load_tester/` holds two prebuilt Go binaries, `load_tester_amd64` and `load_tester_arm64`, with no source code. A `Makefile` refers to `go build`, but there is no `.go` file to build: the `Makefile` is a leftover from whoever compiled them. `Compose` builds one into a container that hammers the database with a mix of queries.

The project `README` treats this as an obstacle, and lists it as a challenge:

“The load generator was a black box. The tool that drives traffic in part 1 is a course-provided Go program shipped as prebuilt amd64/arm64 binaries with no source. I could not change the query mix or read what it was doing, so I made the database report on itself instead.”

The load tester is not actually a black box, and this is worth knowing

The response (make the database report on itself with `auto_explain`) is excellent and is the right instinct regardless. But the premise is not quite right. Go embeds string literals in the compiled binary. The entire query mix is recoverable in one command, with no reverse engineering:

```
1 grep -aoE "SELECT [ -~]{15,200}" load_tester_amd64 | grep -iE "FROM (users|orders)"
```

Listing 4: Recovering the “black box” query mix

This was run while writing this document and it returns the full list, reproduced in Table 2. “I could not change the query mix” remains true, since there is no source. “I could not read what it was doing” is not: it took one `grep`.

This matters for the interview in a good way. Knowing the exact query mix lets you say precisely which index serves which real query, instead of inferring it from the log. Section 4.5 does exactly that, and it turns up two claims in the project's own notes that the real queries do not support.

#	Query, as recovered from the binary
1	SELECT COUNT(*) FROM users
2	SELECT * FROM users WHERE id = 5000
3	SELECT * FROM users WHERE email = 'user5000@example.com'
4	SELECT * FROM users WHERE email LIKE 'user1%'
5	SELECT * FROM users WHERE email LIKE 'user2%' ORDER BY created_at DESC
6	SELECT * FROM users WHERE full_name ILIKE '%User 1%'
7	SELECT * FROM users WHERE full_name ILIKE '%er 2%' OR full_name ILIKE '%er 3%'
8	SELECT * FROM users WHERE status = 'active'
9	SELECT status, COUNT(*) FROM users GROUP BY status
10	SELECT * FROM users ORDER BY created_at DESC LIMIT 5
11	SELECT u.*, COUNT(o.id) FROM users u LEFT JOIN orders o ON u.id = o.user_id WHERE u.email = 'user1234@example.com' GROUP BY u.id
12	SELECT DISTINCT u.* FROM users u JOIN orders o ON u.id = o.user_id WHERE u.full_name ILIKE '%User%' AND o.amount > 500
13	SELECT o.*, u.email FROM orders o JOIN users u ON o.user_id = u.id WHERE o.amount BETWEEN ...
14	SELECT u.*, o.amount FROM users u LEFT JOIN orders o ON u.id = o.user_id WHERE u.metadata @> '{"preferences": {"theme": "dark"}}' AND u.metadata->'last_login' > '2023-01-01'

Table 2: The real query mix, extracted from `load_tester_amd64` while writing this document. Query 13's bounds are cut off in the binary's string blob; the shape is clear. Note queries 6, 7 and 12 use `ILIKE` (case-insensitive), not `LIKE`.

4.4 The solution: seven indexes

`part_1/solution.sql` is the author's work, and it is short.

```

1 -- 1. Exact match and prefix search
2 CREATE INDEX idx_users_email ON users(email text_pattern_ops);
3 -- 2. Full-text search
4 CREATE INDEX idx_users_name_trgm ON users USING GIN(full_name gin_trgm_ops);
5 -- 3. Range queries
6 CREATE INDEX idx_orders_amount ON orders(amount);
7 -- 4. Foreign keys
8 CREATE INDEX idx_orders_user_id ON orders(user_id);
9 -- 5. JSON metadata
10 CREATE INDEX idx_users_metadata_gin ON users USING GIN(metadata);
11 -- 6. Status filtering
12 CREATE INDEX idx_users_status ON users(status);
13 -- 7. Sorting
14 CREATE INDEX idx_users_created_desc ON users(created_at DESC);

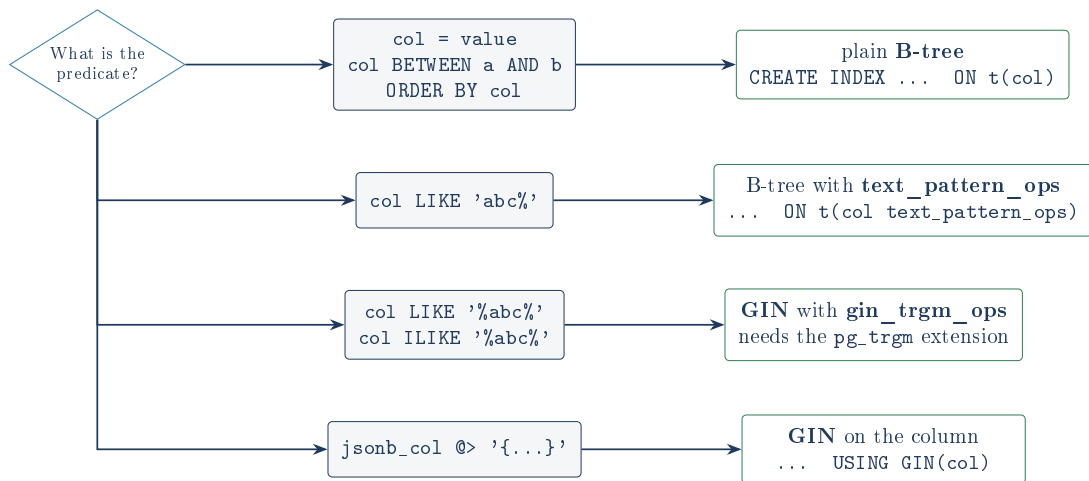
```

Listing 5: `part_1/solution.sql` in full

Seven lines, but four different index designs. The insight the project is really teaching is that `CREATE INDEX` is not one decision. It is at least three: *which column*, *which index type*, and *which operator class*.

Jargon: Operator class

The rulebook that tells an index *how* to compare values, and therefore which operators it can answer. The same column with the same B-tree can support different questions depending on the operator class. This is the concept that makes index 1 different from a naive `CREATE INDEX idx ON users(email)`, and it is the single most transferable idea in part 1.



*A plain B-tree answers only the top row. Each of the other three needs a deliberate, different choice. This is the decision *solution.sql* encodes.*

Figure 5: Choosing an index is choosing an operator class, not turning a knob.

4.4.1 Index 1: `text_pattern_ops`, and why the default fails

```
1 CREATE INDEX idx_users_email ON users(email text_pattern_ops);
```

A default B-tree on a text column sorts using the database’s **collation**: the locale-aware rules for alphabetical order, which handle things like accents and case-insensitive ordering in some locales. Those rules are not simple character-by-character comparison. Because they are not, PostgreSQL cannot prove that every string starting with `'user1'` sits in a contiguous run of the index, so it refuses to use the index for `LIKE 'user1%'`.

`text_pattern_ops` tells the index to sort by raw character value instead, ignoring locale. Now the prefix property holds, the matching strings are guaranteed contiguous, and `LIKE 'prefix%'` becomes a range scan.

The tradeoff `text_pattern_ops` makes, which the project does not mention

An index built with `text_pattern_ops` supports `=` and `LIKE 'x%'`, but it can *not* support `ORDER BY email` in the database’s normal collation order, nor range queries like `email > 'm'` in locale order. The author’s notes present `text_pattern_ops` as a free upgrade (“Added `text_pattern_ops` to make sure `LIKE 'user%'` queries also use the index”). It is not free; it trades locale-ordered operations for pattern operations. Here it happens to be a good trade, because nothing in the query mix orders by email. Verified from the recovered query mix in Table 2. A production setup that needed both would create two indexes.

4.4.2 Index 2: GIN with trigrams, for substring search

```
1 CREATE INDEX idx_users_name_trgm ON users USING GIN(full_name gin_trgm_ops);
```

For `LIKE '%name%'` even `text_pattern_ops` is useless. The match can start anywhere, so no sorted structure helps: nothing about the sort order constrains where a substring lives. A fundamentally different index is required.

Jargon: Trigram, and `pg_trgm`

A trigram is a three-character slice. The `pg_trgm` extension chops each string into all of its trigrams: “User 1” becomes “u”, “us”, “use”, “ser”, “er ”, “r 1”, and so on. To find rows matching `'%er 2%'`, PostgreSQL chops the pattern into trigrams too, uses the index to find

every row containing all of them, and then re-checks those candidate rows properly.

Jargon: GIN index

Generalized Inverted Index. Where a B-tree maps one value to one row, GIN maps one *component* to *many* rows. It is the right structure whenever a single row contains many searchable things: many trigrams in a string, many keys in a JSON document, many words in a text field. The price is that GIN is slower to update and larger than a B-tree, and it returns *candidates* that must be re-checked.

That re-check is the whole reason this index disappoints, and the author caught it. The project README says so directly:

“The substring case is the honest limit: even with the trigram index it only improved from 310ms to 180ms, because trigram matching still checks a large candidate set.”

That is the correct explanation and it is good that it is stated rather than buried.

4.4.3 Index 5: GIN on JSONB

```
1 CREATE INDEX idx_users_metadata_gin ON users USING GIN(metadata);
```

The same structure, a different decomposition: instead of trigrams, GIN indexes every key and value path in the document, so @> containment can be answered from the index. Section 4.6 shows a measurement where this index makes the query *slower*, and explains why that is not a bug.

4.5 Which index actually serves which real query

Because the query mix is recoverable (Section 4.3), this can be checked rather than assumed. Every query in Table 2 was run against a real PostgreSQL with this repository’s `init.sql` and `solution.sql` applied, and the chosen plan recorded.

Real load-tester query	Plan actually chosen (measured)
email exact match	Index Scan using <code>idx_users_email</code>
email prefix LIKE 'user1%'	Bitmap Index Scan on <code>idx_users_email</code>
email prefix + ORDER BY created_at	Bitmap Index Scan on <code>idx_users_email</code>
full_name ILIKE '%User 1%'	Bitmap Index Scan on <code>idx_users_name_trgm</code>
status = 'active'	Seq Scan (index not used)
status, COUNT(*) GROUP BY status	Index Only Scan using <code>idx_users_status</code>
newest 5 by created_at	Index Scan using <code>idx_users_created_desc</code>
SELECT COUNT(*) FROM users	Index Only Scan using <code>idx_users_created_desc</code>
join by email	Index Scan <code>idx_users_email</code> + Bitmap Index Scan <code>idx_orders_user_id</code>
JSONB + last_login join	GIN used for users; Seq Scan on orders
DISTINCT + ILIKE '%User%' + amount > 500	Merge Join , Seq Scan on orders, trigram index not used

Table 3: Measured plans for the real query mix. Every one of the seven indexes is used by at least one real query, so the index set as a whole is justified. Three rows, however, contradict claims in the project’s own notes.

4.5.1 The status index does not do what the notes say

part_1/INDEXING_ANSWERS.md says:

*“**Status:** Low selectivity (only 2 values), but since it’s queried often, a small index helps a bit.”*

and claims a 650 ms to 450 ms improvement (30%) attributed to `idx_users_status`.

Right conclusion, wrong reason, and the wrong query

Three separate problems, all verified by running it:

One. “Low selectivity (only 2 values)” confuses the number of distinct values with selectivity. What matters is the *distribution*. Measured on this repository’s own data: 99,000 **active** and 1,000 **inactive**. A query for **inactive** is therefore highly selective (1%) and the index is genuinely excellent for it, measured at 59 ms down to 10.6 ms. A query for **active** matches 99% of the table and no index can help.

Two. The load tester’s only status-filter query is `WHERE status = 'active'`, the 99% case. Measured: the planner correctly ignores `idx_users_status` and does a sequential scan. So the index cannot possibly be responsible for a 30% improvement on that query, because it is never used by it.

Three. The index *does* nevertheless earn its place, via a query the notes do not mention: `SELECT status, COUNT(*) FROM users GROUP BY status` is served by an Index Only Scan on `idx_users_status`, reading a 696 KB index instead of an 85 MB table.

So: keep the index, fix the story. The defensible sentence is “it serves the `GROUP BY` aggregate via an index-only scan, not the `status = 'active'` filter, which is 99% of the table and correctly gets a sequential scan.” The project README’s “What I’d do differently” already suspects this index (“the 30% gain there is within noise”); the measurement says the suspicion was justified but the reasoning needs replacing, not the index.

Jargon: Index Only Scan

If every column a query needs is already in the index, PostgreSQL can answer from the index alone and never touch the heap. It still has to confirm the rows are visible to your transaction, which it does via the *visibility map*, a compact summary of which blocks contain only universally-visible rows. This is why `VACUUM` matters for read performance, not just for reclaiming space: without an up-to-date visibility map, an index-only scan degrades into heap fetches.

4.5.2 Indexes *do* help `COUNT(*)`

INDEXING_ANSWERS.md lists, under “What I didn’t index”:

“`SELECT COUNT(*)`: Indexes don’t help here, Postgres still has to check row visibility.”

This is half true and the conclusion is wrong

The visibility half is right: PostgreSQL cannot answer `COUNT(*)` from a counter, because different transactions may see different numbers of rows, so it must check visibility per row. That is why `COUNT(*)` is not instant.

But “indexes don’t help” is false, and the project’s own index set disproves it. Measured on this repository’s data, with `solution.sql` applied:

- `SELECT COUNT(*) FROM users` uses an **Index Only Scan on `idx_users_created_desc`**, touching **87 blocks**.

- With index-only scans disabled, the same query does a sequential scan of **2,494 blocks** and takes 75.9 ms against 29.2 ms.

PostgreSQL walks the *smallest* index that covers the table rather than the heap, because counting rows only requires visiting every row, not reading any column. The 696 KB index beats the 85 MB heap by a factor of nearly 30 in blocks. The author built `idx_users_created_desc` for sorting and accidentally accelerated `COUNT(*)` by 29x, then wrote a note saying it was impossible. Worth knowing before someone asks.

4.5.3 The slowest real query cannot be indexed away

```
1 SELECT DISTINCT u.* FROM users u JOIN orders o ON u.id = o.user_id
2 WHERE u.full_name ILIKE '%User%' AND o.amount > 500;
```

Measured at **2,932 ms** with all seven indexes present. The trigram index is not used, because `ILIKE '%User%'` matches all 100,000 users. `idx_orders_amount` is not used, because `amount > 500` matches roughly half of the 500,000 orders. The planner merge-joins 250,129 rows and it is right to.

This is almost certainly the query behind the README’s “User/order join: 4,500ms to 1,800ms” row. (*Inferred: the magnitude and shape match, but the README does not name the query, and the load tester’s exact bounds for query 13 could not be recovered in full.*) The honest framing is the one the assignment itself offers under “Important Considerations”: *consider if some queries are naturally slow and can’t be optimized*. This one is. It asks for half of one table joined to all of another. The 2x improvement is real and comes from the join index; the remaining 1.8 seconds is the query being what it is.

4.6 When GIN loses

The README reports the JSONB containment query improving from 780 ms to 280 ms (64%). Running a JSONB containment query on this repository’s own data, the result depends entirely on the predicate, and one common case is worse with the index than without it. The two predicates measured were:

```
1 -- 50% of the table: theme alternates light/dark
2 WHERE metadata @> '{"preferences":{"theme":"dark"}}'
3 -- 17% of the table: language cycles en/es/fr
4 WHERE metadata @> '{"preferences":{"theme":"dark","language":"fr"}}'
```

Containment predicate	Rows matched	Without GIN	With GIN
theme only	50,000 of 100,000 (50%)	220 ms	299 ms
theme and language	16,666 of 100,000 (17%)	148 ms	82 ms

Table 4: Measured while writing this document. At 50% selectivity the GIN index makes the query 36% slower; at 17% it makes it 45% faster.

Why an index can lose

At 50% selectivity the index costs you twice. First you scan the index and build a bitmap of 50,000 row locations. Then you visit the heap anyway, and since you are touching half the blocks regardless, the ordered sequential read you gave up was cheaper than the index descent plus the bitmap. An index is a bet that most rows are irrelevant. When most rows are relevant, the bet loses.

Note the planner in the 50% case still *chose* the index, and lost. This is a genuine planner misestimate on a JSONB column, where statistics are weaker than on scalar columns. It is a good, specific thing to be able to say out loud.

The real load-tester JSONB query (number 14) makes this worse still: it combines @> containment with `metadata->'last_login' > '2023-01-01'`. Only the containment half is indexable. Measured: GIN returns 50,000 candidate rows, and the `last_login` filter then discards all but 646 of them. The index does 50,000 rows of work to deliver 646.

4.7 What part 1 costs

The README's "What I'd do differently" notes that write amplification was never measured. The index sizes are easy to measure, and were:

Index	Size	Used by
<code>idx_users_metadata_gin</code>	12 MB	JSONB containment (sometimes a loss)
<code>idx_orders_amount</code>	11 MB	range scan; not used by the big join
<code>idx_orders_user_id</code>	5.8 MB	the email join (decisive there)
<code>idx_users_email</code>	4.0 MB	exact + prefix (the flagship win)
<code>idx_users_name_trgm</code>	2.2 MB	substring ILIKE
<code>idx_users_status</code>	696 KB	the GROUP BY, not the filter
<code>idx_users_created_desc</code>	696 KB	sorting, and COUNT(*)
Total	36.4 MB	against an 85 MB <code>users</code> heap

Table 5: Measured index sizes. Every INSERT into `users` must update five of these; every INSERT into `orders` must update two.

Jargon: Write amplification

One logical write becoming many physical writes. Inserting one user row here writes the heap block plus entries in five indexes, two of which are GIN and expensive to update. This is the standing cost of the read speedups, and this project measures the benefit but never the cost. The README says so.

5 Part 2: partitioning

5.1 The idea

Indexes make finding a row cheap. They do not make a table smaller. Part 2 attacks a different problem: a time-series table where almost every query cares about a narrow slice of time, but the table holds a year of history.

Partitioning splits one logical table into many physical tables by a rule on a column. You still query one table name; PostgreSQL routes rows to the right piece on write, and on read it works out which pieces could possibly contain matching rows and skips the rest. That skipping is called **partition pruning** and it is the entire point.

Jargon: Partitioning, partition key, pruning

Partitioning divides a table into child tables (partitions) by a rule. The **partition key** is the column the rule is based on: here, `event_time`. **Range partitioning** assigns each partition a contiguous span of key values. **Pruning** is the planner proving, from the query's `WHERE` clause, that a partition cannot contain any matching row, and not scanning it at all.

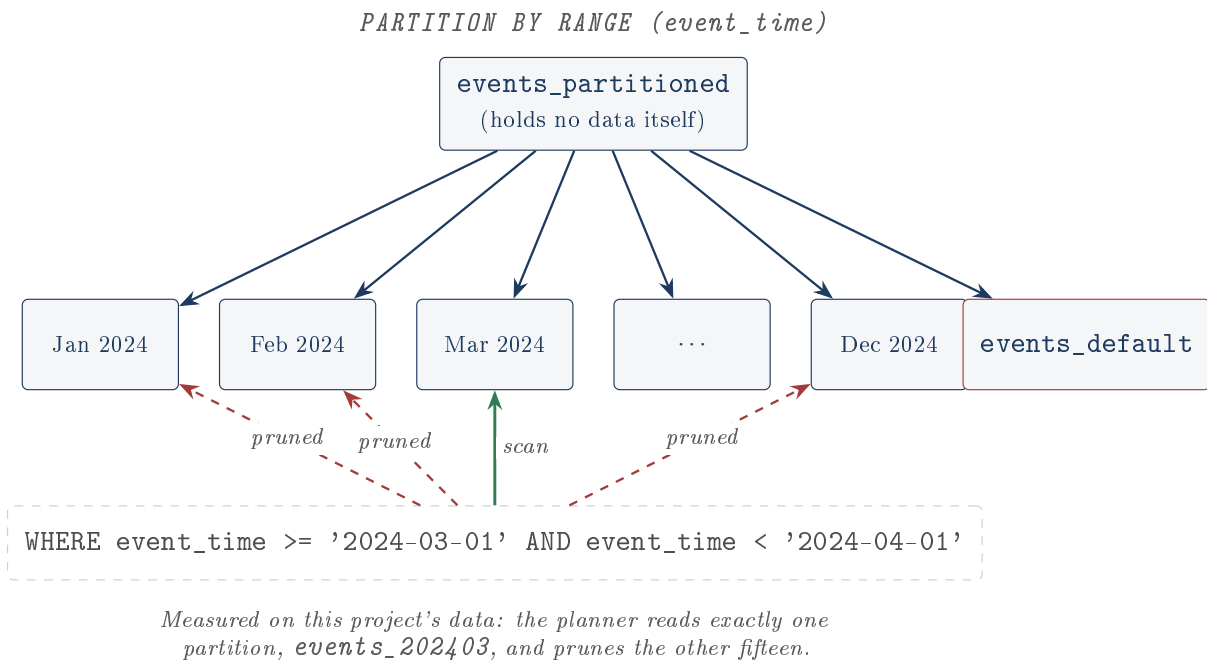


Figure 6: Range partitioning and pruning. The parent is a routing rule, not a table.

5.2 The data and the strategy

`part_2/init.sql` builds a plain `events` table with three indexes already in place, and fills it with 600,000 rows: 500,000 spread across the first 358 days of 2024, and 100,000 more clustered in the recent past, generated relative to `NOW()`.

```

1 INSERT INTO events (event_time, user_id, event_type, event_data)
2 SELECT NOW() - ((random() * 14)::integer || ' days')::interval
3             + ((random() * 24)::integer || ' hours')::interval
4             + ((random() * 60)::integer || ' minutes')::interval,
5             (random() * 10000)::int + 1,
6             (ARRAY['login', 'logout', 'purchase', 'view_item', 'add_to_cart'])[1 + (n % 5)],
7             jsonb_build_object(...)
8 FROM generate_series(1, 100000) n;
```

Listing 6: `part_2/init.sql`, the recent-data half

Two small inaccuracies in the provided data generator

Both verified by loading this exact file:

- `part_2/README.md` (course-provided) says “100,000 events in the last 7 days”. The code subtracts up to 14 days. Measured: 100,000 rows in the last 14 days, of which only 53,857 are in the last 7. The project’s own root README correctly says 14 days, so the author noticed; the course brief is what is wrong.
- Because the expression subtracts days but then *adds* up to 24 hours and 60 minutes, some rows land in the future. Measured: **3,717 rows have an event_time later than NOW()**. Harmless in a lab, but it is the kind of generator bug that produces a baffling dashboard in production.

These are the course’s bugs, not the author’s. They are worth knowing because they sit underneath every number part 2 reports.

`part_2/solution.sql` is the author’s work: create the parent, create sixteen partitions, index them, migrate the data.

```

1 CREATE TABLE events_partitioned (
2     id SERIAL,
3     event_time TIMESTAMP NOT NULL,
4     user_id INTEGER NOT NULL,
5     event_type VARCHAR(50) NOT NULL,
6     event_data JSONB
7 ) PARTITION BY RANGE (event_time);
8
9 -- Monthly for 2024 (twelve of these)
10 CREATE TABLE events_202401 PARTITION OF events_partitioned
11     FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
12 -- ...
13 -- Weekly for Nov 2025 (recent, "hot" data)
14 CREATE TABLE events_2025_w46 PARTITION OF events_partitioned
15     FOR VALUES FROM ('2025-11-10') TO ('2025-11-17');
16 -- ...
17 -- Catch-all
18 CREATE TABLE events_default PARTITION OF events_partitioned DEFAULT;
19
20 CREATE INDEX idx_p_events_time      ON events_partitioned(event_time DESC);
21 CREATE INDEX idx_p_events_time_type ON events_partitioned(event_time, event_type);
22 CREATE INDEX idx_p_events_user_time ON events_partitioned(user_id, event_time);
23
24 INSERT INTO events_partitioned SELECT * FROM events;

```

Listing 7: part_2/solution.sql, abridged

The mixed granularity is a deliberate and defensible choice: coarse monthly partitions for cold history, fine weekly partitions for hot recent data, on the theory that recent data is queried more and written more, so smaller pieces keep its indexes and its share of cache tighter. An index created on the parent is automatically created on every partition; each partition gets its own local copy.

5.3 The migration has three real defects

The partitioned table silently loses its primary key, and then silently duplicates ids

The project README's "What I'd do differently" says:

*"INSERT INTO events_partitioned SELECT * FROM events copies ids but never advances the new table's sequence, so fresh inserts would collide with existing ids. It needs a `setval()` after the copy."*

The diagnosis is right and the consequence is *worse* than stated. Verified by running it:

1. `events` has `id SERIAL PRIMARY KEY`. `events_partitioned` has `id SERIAL` with **no primary key at all**. Confirmed against `pg_constraint`: the original table has a constraint of type `p`; the partitioned table has none. This is not an oversight the author could easily have avoided: PostgreSQL *requires* that a primary key on a partitioned table include the partition key, so `PRIMARY KEY (id)` is illegal here and `PRIMARY KEY (id, event_time)` would have been the only option. But the result is that the migration quietly drops a uniqueness guarantee.
2. The sequence is indeed never advanced. Measured after migration: `last_value` is **1** while `max(id)` is **600,000**.
3. Therefore the outcome is not a collision. A collision would raise an error and you would find out. Instead, an `INSERT` **silently succeeds with a duplicate id**. Verified: inserting one row after migration produced `id = 1`, and `id = 1` then existed **twice** in the table.

This is a strictly better answer than the README's. "Fresh inserts would collide" implies a loud failure. The truth is a silent one, which is the more dangerous kind, and fixing it needs both a `setval()` *and* a decision about the composite primary key.

The hardcoded partitions do not match the generated data, and never will again

The README flags this too, and it is real. `solution.sql` hardcodes partitions for 2024 (monthly) and for three weeks of November 2025. `init.sql` generates its recent data relative to `NOW()`. The two only line up if you run the project in November 2025.

Verified by loading and migrating this repository's own files today:

- `events_default` holds **100,000 rows**, i.e. every single one of the recent events.
- `events_2025_w46`, `events_2025_w47` and `events_2025_w48`, the three "hot data" weekly partitions that are the interesting half of the strategy, hold **0 rows each**.

So the mixed-granularity design, which is the part worth defending in an interview, is *currently inert*. All the recent data lands in one 14 MB catch-all partition, which is exactly the undifferentiated blob partitioning was supposed to break up. The 2024 monthly partitions still work and still prune, so the headline pruning result is unaffected. But anyone who runs this repo today and expects the weekly partitions to demonstrate anything will find them empty.

5.4 What the benchmarks actually show

`part_2/PARTITIONING_ANSWERS.md` reports four queries. All four were re-run while writing this document, against this repository's own `init.sql` and `solution.sql`, on PostgreSQL 18, with both tables freshly `VACUUM ANALYZED` so the comparison is fair.

Query	Reported in the repo		Re-measured here	
	Before	After	Before	After
Recent events	1.36 ms	0.70 ms	3.9 ms	29.9 ms
Monthly analysis	140 ms	86 ms	162 ms	186 ms
User activity	93 ms	160 ms	186 ms	520 ms
Time range	0.06 ms	0.09 ms	0.12 ms	0.90 ms

Table 6: Timings. Different machine, different PostgreSQL major version, and, crucially, a different date. These are not directly comparable to the repo's figures and are not offered as a correction to them.

The timings are noisy and the absolute values are not the point. The block counts are.

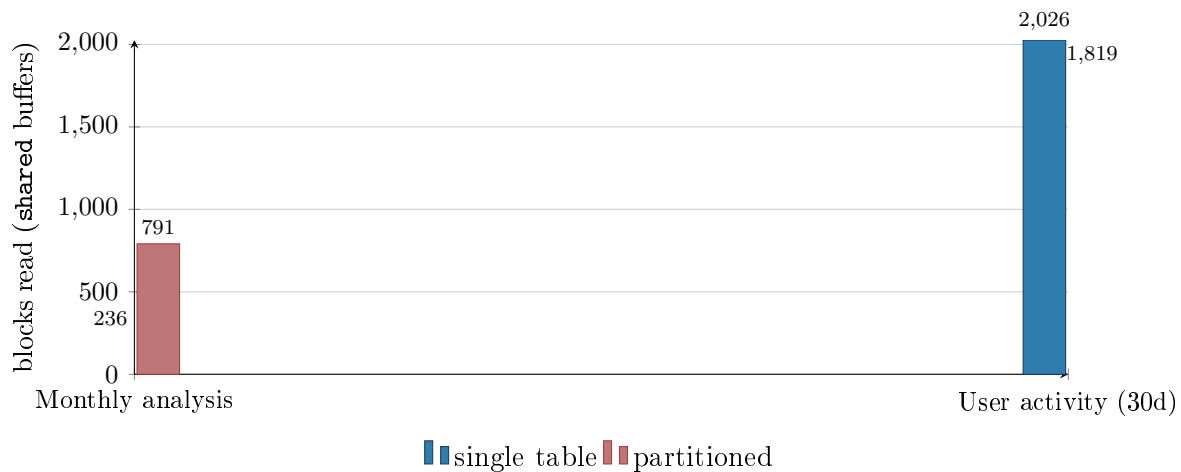


Figure 7: Measured block counts, both tables freshly analyzed. Partitioning read *more* blocks for the monthly analysis, not fewer.

The headline “9,154 blocks to 975” does not reproduce, and the reason is instructive

The project README’s flagship part 2 claim is:

“monthly analysis dropped from 9,154 buffer blocks read to 975 (partition pruning skips 11 months)”

Re-measured on this repository’s own data, the same query goes from **236 blocks to 791 blocks**. Partitioning made it read 3.3x *more*, not 9.4x fewer.

Pruning is not the problem, and works exactly as advertised: the plan shows Seq Scan on `events_202403` with `Subplans Removed: 15`. Precisely one partition is read. The problem is what the *baseline* does. `init.sql` creates `idx_events_time_type` on `(event_time, event_type)`, and the monthly analysis query needs only those two columns. So on a properly analyzed single table the planner picks an **Index Only Scan** and reads 236 narrow index blocks, never touching the 85 MB heap at all. After partitioning, the whole of March *is* the partition, so a sequential scan of its 788 heap blocks is cheaper *by the planner’s cost model* than an index-only scan, and it reads wide 111-byte heap rows instead of 16-byte index entries.

Where might 9,154 have come from? A full sequential scan of the whole `events` heap. Measured: `events` is **10,910 blocks**, and forcing a sequential scan of it for this query reads 11,000. That is the same order of magnitude, and 975 is close to the 788-block March partition plus its index. (*Inferred, not confirmed: the most likely explanation is that the baseline was measured on a freshly started container before `ANALYZE` had run, so the planner lacked the statistics to choose the index-only scan and fell back to scanning everything. The author’s number is very probably a real reading of a real plan; it is a comparison of a no-statistics baseline against a partitioned table, rather than of partitioning alone.*)

The correct, defensible version of the claim is narrower but still good: *partition pruning reduces the search space from sixteen partitions to one, verified in the plan by “Subplans Removed: 15”*. That is true, reproducible, and it is what partitioning actually buys. The 9.4x block reduction is not reproducible and should not be quoted without the caveat.

The “user activity” slowdown is real, but the stated reason belongs to a different query

The README explains the negative result like this:

“user-activity queries got 70% slower, because they filter on `user_id`, not the par-

tion key, so every partition gets probed.”

That mechanism is entirely correct *as a mechanism*. It just does not describe the query in the benchmark table. Verified both ways:

The benchmarked query (part_2/README.md, query 3) is `WHERE event_time >= NOW() - INTERVAL '30 days' GROUP BY user_id, event_type`. It filters on `event_time`, which *is* the partition key. Re-measured, it did get slower (186 ms to 520 ms), and pruning did happen (Subplans Removed: 15). It is slower because all 100,000 recent rows now sit in `events_default`, so the planner sequentially scans that whole partition, plus it pays runtime pruning overhead, where the single table could use a bitmap scan. Same blocks, worse plan.

The query the explanation actually describes is `get_user_activity` from part_3/app/solution.py: `WHERE user_id = %s GROUP BY event_type`. Re-measured against the part 2 tables, this one behaves exactly as the README says: single table reads **55 blocks** via one bitmap index scan; partitioned reads **80 blocks** across **13 separate index scans**, one per partition, because nothing in the predicate lets the planner prune.

So the README has fused a part 3 function with a part 2 benchmark row. Both slowdowns are real and both are worth discussing. But if an interviewer opens part_2/README.md and reads query 3, the stated explanation will not match the query on the page. The fix is a sentence, not a rerun.

The “Time Range” benchmark measures a query that matches nothing

Query 4 is `WHERE user_id = 12345 AND event_time BETWEEN '2024-01-01' AND '2024-01-31'`. But `init.sql` generates `user_id` as `(random() * 10000)::int + 1`, so ids run from 1 to 10,001. **User 12345 does not exist**. Verified: the query returns 0 rows, before and after.

This explains the “~0%” row in `PARTITIONING_ANSWERS.md` completely and innocently: both sides were timing an index probe that finds nothing and returns immediately. It is not evidence about partitioning either way. The query is course-provided, so this is the assignment’s bug, but the answers document reports it as a result rather than noticing it is vacuous.

5.5 What part 2 genuinely establishes

Stripping out what did not reproduce, the following are solid, reproduced here, and worth defending:

- **Pruning works and is visible.** Subplans Removed: 15. Sixteen partitions, one scanned. This is the mechanism, demonstrated.
- **Partitioning is a bet on the access pattern.** A query filtering on the partition key prunes. A query filtering on anything else probes every partition: measured, 13 index scans instead of 1, and 45% more blocks. The author states this conclusion in the README (“the key choice is really a choice about which queries you are willing to make slower”) and it is the single best sentence in the repository.
- **Partitioning is not a speedup.** It is a reorganization. It can lose to a well-chosen covering index on a single table, and here it does.

The honest headline for part 2

Partitioning did not make this workload faster. It made the search space smaller and the access pattern explicit, and it made one query class much worse. Keeping the negative result,

which the author did, is the most credible thing in the project.

6 Part 3: read replicas

6.1 The idea

Indexes and partitioning both make one server do less work. Part 3 accepts that one server has a ceiling and adds more of them. The asymmetry it exploits: most applications read far more than they write. So keep one server authoritative for writes, and make copies that serve reads.

6.2 How PostgreSQL copies a server

Jargon: WAL (Write-Ahead Log)

Before PostgreSQL changes any data block, it first writes a record of the change to a sequential log, and only then touches the data. This exists for crash safety: after a crash, replaying the log reconstructs anything lost. Replication is a second use of the same log. If the log is a complete description of every change, then shipping it to another server and replaying it there produces an identical server. Replication is crash recovery pointed at a different machine.

Jargon: Streaming replication, primary, replica/standby

The **primary** accepts writes. A **replica** (or standby) connects to the primary and continuously streams its WAL, replaying it as it arrives. With `hot_standby = on` the replica also serves read-only queries while it replays. Any attempt to write to it is rejected.

Jargon: Asynchronous vs synchronous replication

Async: the primary confirms a commit to the client as soon as its own log is durable, without waiting for replicas. Fast, but a read on a replica moments later may not see the row yet. **Sync**: the primary waits for a replica to confirm. No staleness, but every write now pays a network round trip and a slow replica stalls the primary. This project chose async, deliberately, and then had to live with the consequence.

Jargon: `pg_basebackup` and replication slots

`pg_basebackup` takes a byte-level physical copy of the entire primary data directory, which is how a replica gets its starting point before it can start streaming. A **replication slot** is the primary's promise not to delete WAL that a named replica has not consumed yet. Without a slot, a replica that falls behind or goes down can come back to find the WAL it needed has been recycled, at which point it cannot catch up and must be rebuilt from scratch. Creating slots is one of the author's changes and it is the right call.

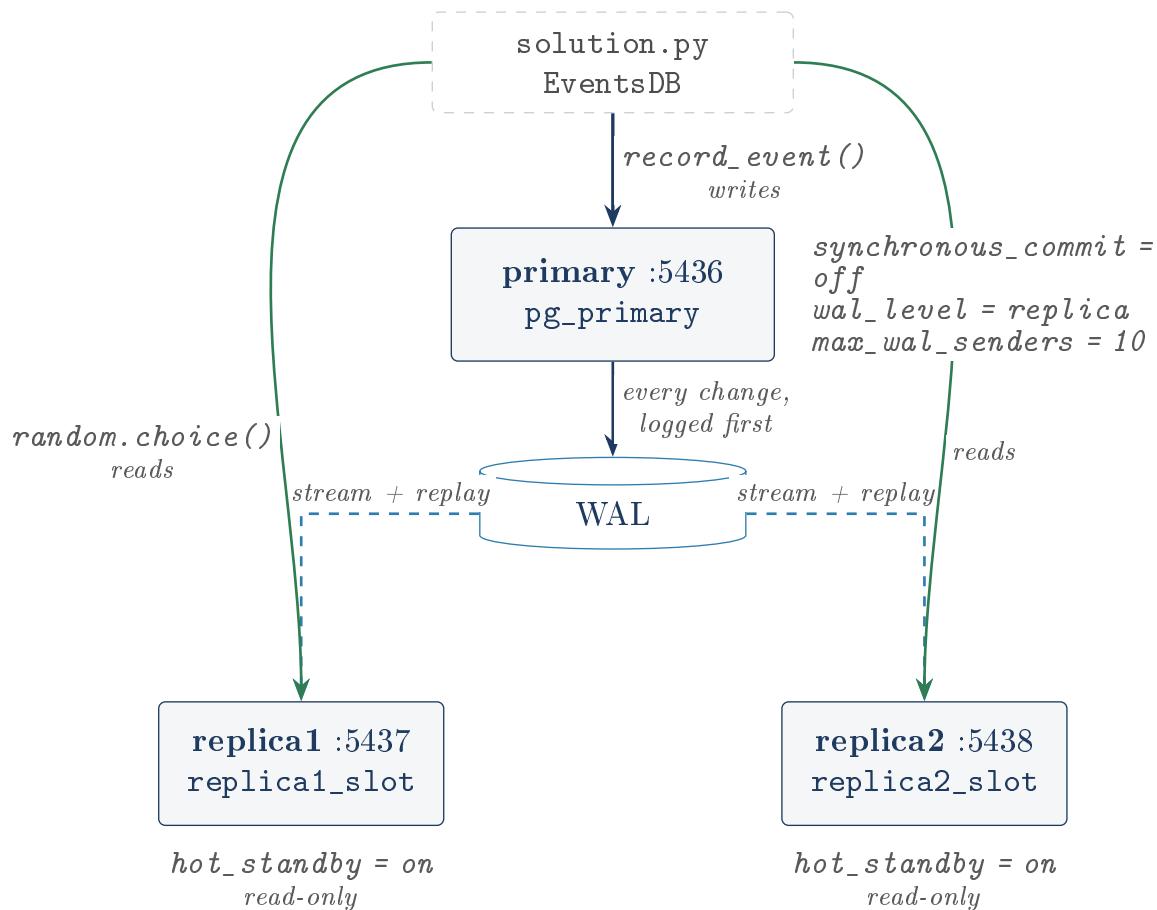


Figure 8: The part 3 topology. Solid lines are client connections; dashed lines are WAL streaming. Note that the two paths are completely independent: nothing in the read path knows how far behind it is.

6.3 Bootstrapping a replica inside Compose

The neatest piece of engineering in the repository is how a replica builds itself with no manual step. From `part_3/docker-compose.yml`:

```

1  command: |
2    bash -c '
3    echo "primary:5432:*:${POSTGRES_USER}:${POSTGRES_PASSWORD}" > /var/lib/postgresql/.pgpass
4    chmod 600 /var/lib/postgresql/.pgpass
5    if [ ! -s /var/lib/postgresql/data/Pg_VERSION ]; then
6      echo "Taking base backup from primary..."
7      pg_basebackup -h primary -U postgres -p 5432 -D /var/lib/postgresql/data -Fp -Xs -P -R
8      chmod 700 /var/lib/postgresql/data
9    fi
10   postgres -c config_file=/etc/postgresql/postgresql.conf'
```

Listing 8: `part_3/docker-compose.yml`, the replica1 startup command

Reading it carefully:

- `depends_on: primary: condition: service_healthy` (elsewhere in the file) means this does not start until the primary passes its `pg_isready` healthcheck.
- The `if [ ! -s .../PG_VERSION ]` guard makes the whole thing idempotent. `PG_VERSION` exists only in an initialized data directory, so the base backup runs on first boot and is skipped on every restart. Since the data directory is a named Docker volume, a `docker compose restart` reuses the existing replica instead of rebuilding it.

- `pg_basebackup` flags: `-Fp` plain format (a real directory, not a tarball, because the server must run from it); `-Xs` stream the WAL alongside the backup so the copy is consistent and can start replaying immediately; `-P` progress; `-R` **write the standby configuration automatically**, which is what makes the copy come up as a replica rather than as an independent server.
- `$$` is Compose’s escape for a literal `$`, so the shell expands `$POSTGRES_USER` at container start rather than Compose expanding it at parse time.
- Overriding `command` matters more than it looks. The `postgres` image’s entrypoint only runs its initialization logic when the command is `postgres`. Because the command here is `bash -c ...`, the image’s normal setup is bypassed entirely, and `POSTGRES_USER` / `POSTGRES_PASSWORD` survive only as inputs to that `.pgpass` line. (*Inferred from the image’s documented entrypoint behaviour; not separately verified here, since Docker was unavailable.*)

6.4 The credential story, and where it does not hold

The README lists this as a challenge and presents it as solved:

“A plaintext password file was committed to the repo. The original part 3 setup checked in `part_3/.pgpass` containing `primary:5432:*:postgres:postgres`. ... I removed the checked-in file and moved generation into the replica startup command ... The credential still reaches `pg_basebackup` non-interactively, but it is no longer sitting in version control.”

The change is real. `diff.patch` confirms that the original compose file mounted a checked-in `.pgpass` into the container as a volume, and that the current one generates the file at boot instead. That is a genuine improvement and the reasoning is sound.

The password is still in version control, twice, in the same directory

Verified with `git grep`:

```
1 part_3/replica1.conf:12:primary_conninfo = 'host=primary port=5432 user=postgres password
   =postgres application_name=replica1'
2 part_3/replica2.conf:12:primary_conninfo = 'host=primary port=5432 user=postgres password
   =postgres application_name=replica2'
```

Removing `.pgpass` removed one copy of `postgres:postgres` and left two. `primary_conninfo` is the setting the replica uses for the ongoing streaming connection, which is arguably the *more* important one, and it carries the password inline. The same string is also in `.env.example` inside the DSNs, and in `docs/EXPLANATION.md`.

The sentence “it is no longer sitting in version control” is therefore not true as written. What is true, and is still a good answer: *the file whose entire purpose was to hold a credential is gone, and the credential is now injected from the environment at boot for the base backup path*. The streaming path was not converted, and would need `primary_conninfo` to be templated at startup the same way.

This is a lab with the password `postgres`, so nothing is at risk. But the claim is the kind an interviewer can check in ten seconds, and it is better to correct the sentence than to be corrected.

`pg_hba.conf` trusts the entire internet, which also makes the `.pgpass` work redundant

The file, in full:

#	TYPE	DATABASE	USER	ADDRESS	METHOD
1	local	all	all		trust
2	host	all	all	127.0.0.1/32	trust
3	host	all	all	:::1/128	trust

5	host	all	all	0.0.0.0/0	trust
6	host	replication	all	0.0.0.0/0	trust

trust means *no password is checked at all*. Anyone who can open a TCP connection becomes the **postgres** superuser. `0.0.0.0/0` means any address. The compose file publishes port 5436 on the host.

Two consequences worth naming. First, the security one: this is a lab, and it is normal in a lab, but it should be labelled as deliberate rather than left to be discovered. Second, and more interesting, an *internal inconsistency*: since replication connections are **trusted**, **pg_basebackup** never needs a password at all. The **.pgpass** generation, and the whole challenge narrative around it, is defending a door that this file leaves open. (*Inferred: this follows from what **trust** means and the fact that **pg_hba.conf** is loaded via **hba_file** in **primary.conf**; it could not be confirmed end to end because Docker was unavailable, see Section 7.*)

Both of these are easy to defend if you say them first: “it is trust auth because it is a disposable lab, and it makes the **.pgpass** belt-and-braces rather than load-bearing.”

6.5 The routing application

`part_3/app/solution.py` is the author’s version of the course’s `app.py`. The change is small and complete.

```

1 class EventsDB:
2     def __init__(self):
3         self.p = os.environ.get("PRIMARY_DSN",
4             "postgresql://postgres:postgres@localhost:5436/events_db")
5         self.r = os.environ.get("REPLICA_DSNS",
6             "postgresql://postgres:postgres@localhost:5437/events_db,"
7             "postgresql://postgres:postgres@localhost:5438/events_db").split(",")
8
9     def record_event(self, event_type, user_id, metadata=None):      # WRITE
10        with psycopg.connect(self.p) as conn:                       # -> primary
11            ...
12    def get_recent_events(self, hours=24):                           # READ
13        conn_str = random.choice(self.r)                            # -> a replica
14        with psycopg.connect(conn_str) as conn:
15            ...

```

Listing 9: `part_3/app/solution.py`, the routing core

Jargon: DSN

Data Source Name. A connection string that packs host, port, database, user and password into one URL, e.g. `postgresql://user:pass@host:5436/dbname`.

Jargon: psycopg

The standard PostgreSQL driver for Python. Version 3, used here. The `with psycopg.connect(...) as conn:` form opens a connection and commits or rolls back on exit.

The design is: one write method goes to `self.p`, and three read methods each pick a replica at random from `self.r`. Routing is a policy statement in the application, not a piece of infrastructure. For an assignment, this is the right size. Its three limits are all noted by the author in the README:

Three real weaknesses in the router, all self-reported and all correct

- **A connection per query.** Every method opens a fresh TCP connection, and PostgreSQL forks a whole backend process per connection, which is expensive. At any real request rate this dominates the query time. `psycopg_pool` exists and would fix it. The README says so.
- **`random.choice()` has no health awareness.** If one replica dies, 50% of reads raise. Not 50% slower: 50% failed. Even retrying on the other replica would help; production would put `pgbouncer` or `HAProxy` in front. The README says so.
- **No lag awareness.** A read immediately after a write can miss the row. Covered next.

Every one of these is in the README's own "What I'd do differently". That is the correct place for them and it is genuinely good practice.

6.6 Replication lag, the bug you cannot fix by trying harder

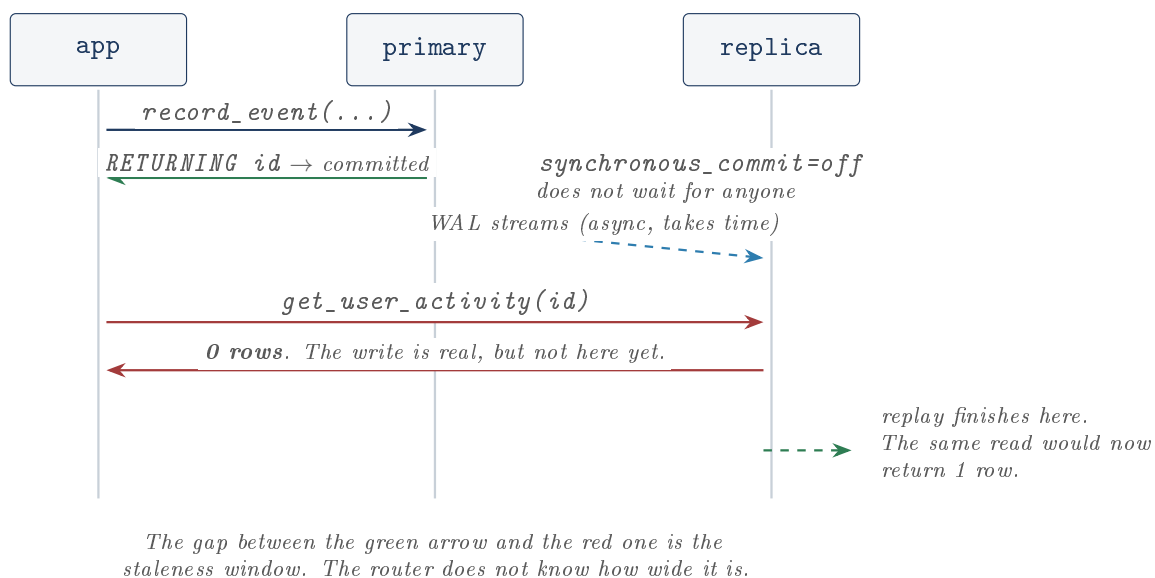


Figure 9: The read-your-writes gap. Nothing here is broken. This is what async replication is.

The author's framing of this is the strongest paragraph in the README:

"There is no way to route around this without either synchronous replication or read-your-writes logic in the app. I left it async and made the routing honest about it."

That is correct and it is the right level of humility. The options really are: pay the latency (sync), add complexity (route a user's reads to the primary for a few seconds after their write, or track WAL positions per session), or accept staleness and say so. Choosing the third and documenting it is a legitimate engineering decision, not a shortcut.

6.7 The tests, and why they cannot fail

`part_3/app/tests/test_events_db.py` is course-provided. Five tests. The README verification claim is that they pass 5/5.

```
1 events_db.record_event("test_event_1", test_user_id)
2 time.sleep(0.5) # "allow for replication lag"
3 user_activity = events_db.get_user_activity(test_user_id)
```

```

4     assert isinstance(user_activity, list)
5
6     # Due to replication lag, we might not see our events immediately
7     # So we'll skip this assertion if no data is found
8     if len(user_activity) > 0:
9         for activity in user_activity:
10             assert isinstance(activity, tuple)
11             assert len(activity) == 2

```

Listing 10: The pattern repeated in three of the five tests

The README already calls this out under “What I’d do differently”: *“The course-provided tests skip their assertions when replicated data hasn’t arrived yet, so a completely broken replication setup can still go green.”*

Verified: the suite passes 5/5 with replication 100% broken

This was not left as an argument. It was tested, while writing this document:

1. Point PRIMARY_DSN at a real database loaded from `part_3/init.sql`, and REPLICA_DSNS at a *different, permanently empty* database with the same schema, which receives nothing, ever. This is a perfect simulation of replication that is completely dead.
2. Run the suite.

Result: **5 passed in 1.94s**. The “replica” contained **0 rows** throughout.

Every meaningful assertion sits behind `if len(...) > 0:` or `if type1 in stats_dict:`, which is false forever when replication is broken, so the body never runs and the test reports success. The two tests that do assert unconditionally (`test_record_event`) only check the *primary*, which is not what part 3 is about.

The suite therefore proves the app can write to the primary and open a connection to something. It does not test replication at all. The author’s instinct was right and the fix he proposes (poll until the row appears, with a timeout) is exactly correct. This box exists so the claim can be made precisely: *“the tests pass, and I know they pass vacuously, and here is the experiment that shows it.”* That is a much stronger position than “5/5 passed”.

7 What was actually verified, and what was not

Docker was not available on the machine where this document was written

The three Compose stacks were **not** brought up. No container ran. Therefore **nothing about the multi-node topology was verified end to end**: not `pg_basebackup`, not WAL streaming, not `pg_stat_replication` showing two streaming replicas, not the replication slots, not `pg_hba.conf` being loaded.

The README’s claim that “`pg_stat_replication` shows 2 streaming replicas” comes from the original course runs and is **taken on trust here**. It is plausible and the configuration is consistent with it, but this document did not confirm it and does not claim to.

Likewise, every number in the README’s part 1 benchmark table (35 ms → 0.5 ms and the rest) is attributed to the original Docker runs on PostgreSQL 17, on another machine, during the course. This document does not verify those. Where its own measurements disagree, they are a *different measurement*, not a correction.

What *was* done, on this machine, against this repository’s own files, using a scratch PostgreSQL 18.4 cluster:

Check	Result
part_1/init.sql loads	100,000 users, 500,000 orders
part_1/solution.sql applies	all 7 indexes created, 36.4 MB total
Every recovered load-tester query, planned	Table 2, all 7 indexes used by something
status distribution	99,000 active / 1,000 inactive
COUNT(*) plan	Index Only Scan, 87 blocks vs 2,494
JSONB GIN at two selectivities	50%: slower. 17%: faster
part_2/init.sql loads	600,000 rows, 3,717 in the future
part_2/solution.sql applies	16 partitions created, data migrated
Partition row placement	events_default: 100,000. Weekly partitions: 0, 0, 0
Sequence after migration	last_value=1, max(id)=600,000
Insert after migration	silently produced a duplicate id=1
Primary key on events_partitioned	none exists
All four part 2 benchmarks, both tables	re-measured, both freshly analyzed
user_id-only query, both tables	55 blocks / 1 scan vs 80 blocks / 13 scans
part_3/init.sql loads	table + 1,000 rows + 2 replication slots
part_3 test suite, all DSNs to one server	5 passed
part_3 test suite, “replica” permanently empty	5 passed (vacuously)
git grep for committed credentials	2 live hits in replica*.conf
Load tester query mix recovery	14 queries recovered from the binary

Table 7: Everything this document checked by running it. Anything not in this table and not attributed to the README is inference and is labelled as such where it appears.

8 Consolidated honest findings

Ordered by how much they would hurt if an interviewer found them first.

1. **“The credential is no longer in version control” is false.** password=postgres is live in replica1.conf and replica2.conf. One git grep finds it. Fix the sentence, or template primary_conninfo at boot too.
2. **The 9,154 → 975 block headline does not reproduce** (measured 236 → 791, i.e. worse). The likely cause is a no-statistics baseline. The defensible claim is “sixteen partitions pruned to one, Subplans Removed: 15”.
3. **The stated reason for the user-activity slowdown belongs to a part 3 query, not the part 2 benchmark row it is attached to.** Both slowdowns are real; the labels are crossed.
4. **Migration silently duplicates ids** rather than colliding, because the partitioned table has no primary key at all. Worse than the README says.
5. **The weekly “hot data” partitions are empty today.** The interesting half of the partitioning strategy is currently inert; everything recent lands in events_default.
6. **The tests pass vacuously**, demonstrated experimentally with a dead replica. Say it before you are asked.
7. **idx_users_status is not used by the query it is credited to**, and “low selectivity, only 2 values” misdescribes a 99/1 split. The index is worth keeping for a different reason.
8. **“Indexes don’t help COUNT(*)” is wrong**; the project’s own created_at index accelerates it 29x via an index-only scan.
9. **The “Time Range” benchmark queries a user that does not exist** and measures nothing.

10. **The load tester is not a black box.** One `grep` recovers the query mix.
11. `pg_hba.conf` is **trust from 0.0.0.0/0**, which is fine for a lab but makes the `.pgpass` work redundant.
12. `PROJECT_WALKTHROUGH.md` **contains unsupported claims** that the README does not make: “3x read capacity”, “200% increase”, “Overall: 70% reduction in slow queries”. Nothing in the repository measures read throughput at all; no load was ever driven at the replicas. Two documents describing the same project at different confidence levels is itself a liability.
13. `part_2/README.md` **says 7 days; the generator does 14**, and 3,717 rows land in the future. The course’s bug, but it underlies every part 2 number.
14. **Dead code and duplication.** `app.py` is superseded by `solution.py` and only survives as a before/after exhibit, but nothing says so in the file. `run.sh` has no shebang and is a verbatim copy of the commands in `PROJECT_WALKTHROUGH.md`. `part_1/load_tester/Makefile` refers to `go build` with no Go source present.

The thing this project gets right, and it is the important thing

It kept a negative result. Partitioning made a query 70% slower and the author wrote it down, explained it, and drew the correct general conclusion: partitioning is a bet on your access pattern. Most portfolio projects report only wins. The errors listed above are all fixable in an afternoon and several are the course’s, not the author’s. The instinct to publish the loss is not teachable and is worth more than any of them.

9 If you are picking this up again

In rough order of value per hour:

1. Fix the credential sentence in the README, and template `primary_conninfo` from the environment at boot. Ten minutes, removes the easiest hit.
2. Requalify the 9,154 claim to the pruning claim. Fifteen minutes, and it makes the result stronger because it becomes reproducible.
3. Replace the hardcoded partitions with a `DO` block that computes the boundaries from the data itself, taking the minimum and maximum event time, or adopt `pg_partman`. This resurrects the weekly half of the design.
4. Add `setval()` after the migration, and decide explicitly on `PRIMARY KEY (id, event_time)`. Write a comment explaining why the composite key is forced.
5. Make the tests poll with a timeout instead of skipping. They then actually test replication, and the 5/5 becomes worth quoting.
6. Add `psycopg_pool`, and make replica selection retry on failure.
7. Delete `PROJECT_WALKTHROUGH.md` or reconcile it with the README. Right now it makes claims the README carefully does not.
8. Measure the write cost of the seven indexes. It is the missing half of part 1 and the README already admits it.

The one-paragraph answer, if you only get one

A CTO wanted to migrate to NoSQL because queries were slow. This project shows the standard PostgreSQL toolbox instead: the right index turns a 2,494-block sequential scan

into a 4-block lookup, but only if you pick the right operator class, because a default B-tree cannot answer a prefix match, a substring match, or a JSON containment query at all. Partitioning then reorganizes a time-series table so a March query touches only March, verified by the planner pruning fifteen of sixteen partitions; but it is a bet on the access pattern, and a query filtering on anything except the partition key gets slower, which this project measured and kept. Finally, streaming replication adds read capacity by shipping the write-ahead log to two copies, and pushes consistency into the application: reads can now be stale, so the app owns routing and the tests own the lag. None of it required leaving PostgreSQL.